

Análisis de Algoritmos 2022/2023

Práctica 1

Íñigo Álvarez y Enrique Gómez, Grupo 1271.

Código	Gráficas	Memoria	Total

1. Introducción.

Vamos a realizar una práctica en la que implementaremos varias funciones y algoritmos y calcularemos su complejidad. Se programará en el lenguaje C y además se usará gnuplot para generar gráficas, midiendo tiempos y comparando datos.

2. Objetivos

El objetivo de esta práctica es aprender a implementar algunos algoritmos, además de medir tiempos y la complejidad de estas funciones. Además aprenderemos a generar gráficas con gnuplot comparando dichos tiempos con su supuesto resultado.

2.1 Apartado 1

El apartado uno tiene como objetivo generar N números aleatorios entre un límite inferior y un límite superior. Con este también comprobaremos que con un número muy grande de N se igualarán las veces que cada número se repite, para ver que realmente es aleatorio y todos tienen la misma probabilidad de salir.

2.2 Apartado 2

En el apartado 2 el objetivo es aprender lo que es una permutación y a realizarlas para generar una array desordenada de N elementos aleatorios menores que un size.

2.3 Apartado 3

En el apartado 3 queremos generar arrays de 0 a N para luego hacer un número determinado de permutaciones para desordenar esa tabla y poder volverla a ordenar en próximos ejercicios.

2.4 Apartado 4

El objetivo del apartado 4 es ordenar mediante el algoritmo Selectsort un array que hemos creado desordenado de los números de 1 a N mediante la función del apartado 3 . Con ello aprenderemos el funcionamiento del Selectsort y contaremos el número de veces que se ejecuta la OB para ver si coincide con lo que debería.

2.5 Apartado 5

El objetivo del apartado 5 es medir los tiempos de ejecución de un algoritmo en varias permutaciones, almacenarlos en una tabla junto con otros datos como el número de veces que se ejecuta la operación básica o el número de permutaciones y su longitud, y guardar todos estos datos de cada permutación en un archivo.

2.6 Apartado 6

El objetivo del apartado 6 es crear una función inversa al Selectsort y comparar los resultados generados con el apartado 5 entre las dos funciones.

3. Herramientas y metodología

3.1 Apartado 1

Hemos utilizado el entorno de desarrollo Linux, con Visual Studio Code como herramienta para programar. Para compilar el programa hemos utilizado gcc así como algunas flags como -Wall, -ansi o -pedantic. Valgrind lo hemos empleado para comprobar si la memoria de nuestro programa estaba correctamente usada. Gnuplot ha sido necesario para generar una gráfica con los números random generados y su número de veces. Para comprobar el funcionamiento de este apartado hemos usado el test que hay en el Makefile, cambiando sus datos para comprobar posibles fallos. También al generar la gráfica puedes detectar posibles errores.

3.2 Apartado 2

Como en el apartado anterior también hemos utilizado el entorno de desarrollo Linux y las herramientas Visual Studio Code, gcc o valgrind . Para comprobar si estaba bien se ha usado el test del Makefile (exercise2_test) cambiando los datos en función de lo que quisiéramos comprobar.

3.3 Apartado 3

En este apartado también se ha utilizado el entorno de desarrollo Linux y las herramientas Visual Studio Code, gcc o valgrind. Como pruebas hemos usado el test que hay en el Makefile (exercise3_test) cambiando los datos en función de las pruebas que quisiéramos hacer.

3.4 Apartado 4

Como en todos los apartados anteriores también hemos utilizado el entorno de desarrollo Linux y las herramientas Visual Studio Code, gcc o valgrind. Como pruebas una vez más hemos usado el test del Makefile (exercise4_test) cambiando los datos según nos interesaba. En este apartado además también hemos creado nuestros propios mains para comprobar el proceso del SelectSort con más facilidad, contar si el número de operaciones básicas era correcto e ir viendo cómo se modificaba el array poco a poco.

3.5 Apartado 5

En el apartado 5 también se ha utilizado el entorno de desarrollo Linux y las herramientas Visual Studio Code, gcc o valgrind. También hemos empleado la herramienta Gnuplot para generar gráficas con los datos obtenidos. La prueba principal ha sido el test del makefile (exercise5_test) cambiando los datos para generar distintas pruebas.

3.6 Apartado 6

En este último apartado se ha utilizado el entorno de desarrollo Linux y las herramientas Visual Studio Code, gcc o valgrind. Para comprobar nuestra función hemos modificado las pruebas del apartado 4 llamando a la nueva función en vez de a SelectSort.

4. Código fuente

Aquí ponéis el código fuente **exclusivamente de las rutinas que habéis desarrollado vosotros** en cada apartado.

4.1 Apartado 1

```
int random_num(int inf, int sup)
{
    int n = 0;

    if (inf > sup || inf < 0 || sup < 0)
        return ERR;
    n = rand () % (sup-inf) + inf;

    return n;
}
```

4.2 Apartado 2

```
int* generate_perm(int N)
{
    int i = 0;
    int j = 0;
    int aux = 0;
    int *perm = NULL;

    if (N < 0)
        return NULL;
    perm = (int*)malloc(sizeof(int) * N);
    if (!perm)
        return NULL;

    for (i = 0; i < N; i++)
        perm[i] = i;
    for (i = 0; i < N; i++)
    {
        aux = perm[i];
        j = random_num(i, N);
        if (j == ERR) {
            free(perm);
            return NULL;
        }
        perm[i] = perm[j];
        perm[j] = aux;
    }
    return perm;
}
```

4.3 Apartado 3

```
int** generate_permutations(int n_perms, int N)
{
    int i = 0;
    int **perm = NULL;

    if (N < 0 || n_perms < 0)
        return NULL;
    perm = malloc(sizeof(int*) * n_perms);
    if (!perm)
        return NULL;
    for (i = 0; i < n_perms; i++)
    {
        perm[i] = generate_perm(N);
        if (perm[i] == NULL)
        {
            while (i >= 0)
            {
                i--;
                free(perm[i]);
            }
            free(perm);
            return NULL;
        }
    }
    return perm;
}
```

4.4 Apartado 4

```
int SelectSort(int* array, int ip, int iu)
{
    int i = 0, counter=0, minm=0, swap=0;
    if (!array || iu < ip || ip<0)
        return ERR;
    for (i = ip; i <= iu-1; i++)
    {
        minm = i;
        minm= min(array, i, iu, &counter);
        if (minm == ERR)
            return ERR;
        swap= array[i];
        array[i]= array[minm];
        array[minm]=swap;
    }
    return counter;
}

int min(int* array, int ip, int iu, int *counter)
{
    int min=0, j=0;
    if (!array || iu<ip || ip<0 || !counter){
        return ERR;
    }
    min=ip;
    for(j=ip+1; j<=iu; j++){
        if(array[j]<array[min]){
            min=j;
        }
        (*counter)++;
    }
    return min;
}
```

4.5 Apartado 5

```
short generate_sorting_times(pfunc_sort method, char* file, int num_min, int num_max, int incr, int n_perms)
{
    PTIME_AA time;
    int i, N, j;

    if (!method || !file || num_min < 0 || num_max < 0 || num_min > num_max || incr <= 0 || n_perms < 0)
        return ERR;
    N = ((num_max-num_min)/incr) + 1;
    time = malloc(sizeof(TIME_AA)*N);
    if (!time)
        return ERR;
    for (i = 0, j = num_min; i < N; i++)
    {
        if(average_sorting_time(method, n_perms, j, &time[i])==ERR){
            free(time);
            return ERR;
        }
        j+=incr;
        if (j > num_max + 1)
            break;
    }
    if(save_time_table(file, time, i) == ERR){
        free(time);
        return ERR;
    }

    free(time);
    return OK;
}

short save_time_table(char* file, PTIME_AA ptime, int n_times)
{
    FILE *f;
    int i = 0;

    if (!file || !ptime || n_times <= 0)
        return ERR;
    f = fopen(file, "w");
    if (!f)
        return ERR;
    for (i = 0; i < n_times; i++)
    {
        fprintf(f, "Size: %d\nExecution time: %f\nAverage: %f\nMax: %d\nMin: %d\n", ptime[i].N, ptime[i].time, ptime[i].average_ob, ptime[i].max_ob, ptime[i].min_ob);
    }
    fclose(f);
    return OK;
}
```

```

short average_sorting_time(pf_func_sort metodo, int n_perms, int N, PTIME_AA ptime)
{
    int **permutations;
    int i = 0;
    clock_t start, end;
    int count = 0;
    int count_min = 0;
    int count_total = 0;
    int count_max = 0;
    double a = 0;

    if (!metodo || n_perms < 0 || N < 0 || ptime == NULL)
    | return ERR;
    permutations = generate_permutations(n_perms, N);
    if (permutations == NULL)
    | return ERR;
    for (i = 0; i < n_perms; i++)
    {
        start = clock();
        count = metodo(permutations[i], 0, N - 1);
        if (count == ERR)
        {
            while (i < n_perms)
            {
                free(permutations[i]);
                i++;
            }
            free(permutations);
            return ERR;
        }
        end = clock();
        count_total += count;
        if (count <= count_min || count_min == 0)
        | count_min = count;
        if (count >= count_max)
        | count_max = count;
        a += ((end - start));
    }
    i = 0;
    ptime->time = a / n_perms;
    ptime->n_elems = n_perms;
    ptime->N = N - 1;
    ptime->average_ob = count_total / n_perms;
    ptime->min_ob = count_min;
    ptime->max_ob = count_max;
    while (i < n_perms)
    {
        free(permutations[i]);
        i++;
    }
    free(permutations);
    return OK;
}

```

4.6 Apartado 6

```

int SelectSortInv(int* array, int ip, int iu)
{
    int i = 0, counter=0, maxm=0, swap=0;
    if (!array || iu < ip || ip < 0)
    | return ERR;

    for (i = ip; i <= iu - 1; i++)
    {
        maxm = i;
        maxm = max(array, i, iu, &counter);
        if (maxm == ERR)
        | return ERR;
        swap = array[i];
        array[i] = array[maxm];
        array[maxm] = swap;
    }
    return counter;
}

```

```

int max(int* array, int ip, int iu, int *counter)
{
    int max=0, j=0;

    if (!array || iu < ip || ip < 0 || !counter){
    | return ERR;
    }

    max=ip;
    for(j=ip+1; j<=iu; j++){
        if(array[j]>array[max]){
        | max=j;
        }
        (*counter)++;
    }
    return max;
}

```

5. Resultados, Gráficas

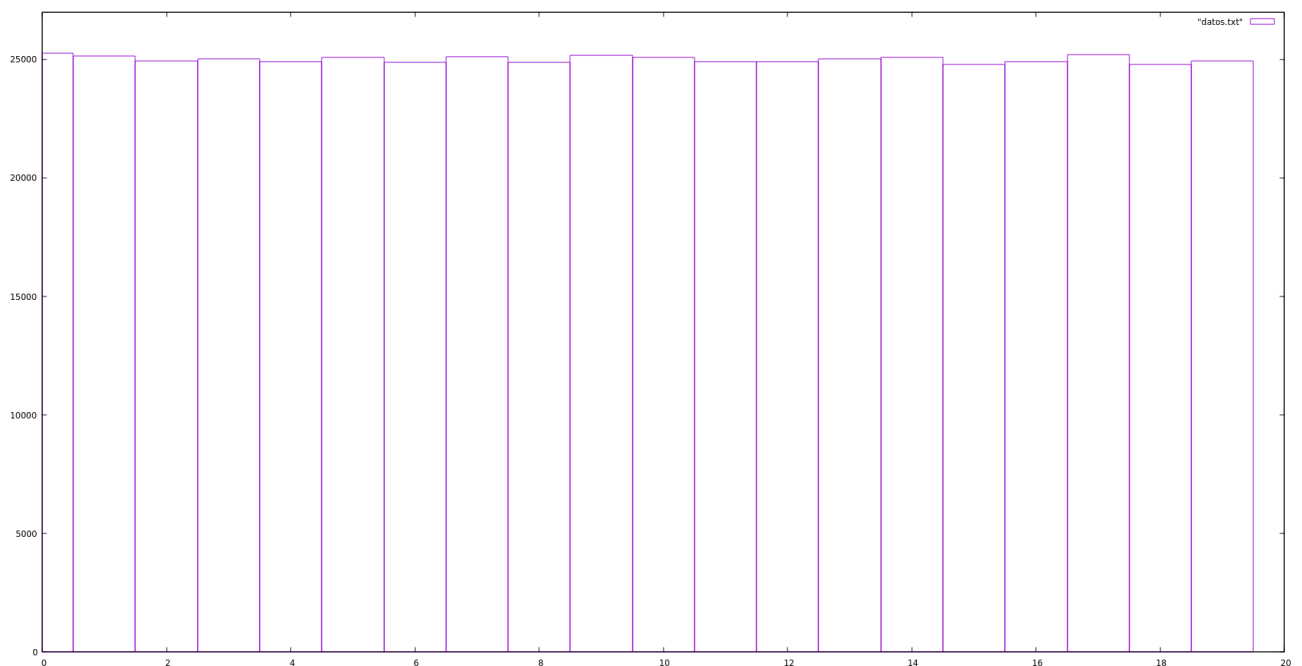
Aquí ponéis los resultados obtenidos en cada apartado, incluyendo las posibles gráficas.

5.1 Apartado 1

Tras ejecutar el ejercicio1.c nos deberían salir x números aleatorios entre un número mínimo y máximo elegido al ejecutar el fichero. Con nuestro código, saldrán los a la izquierda los números que pueden salir y a la derecha el número de veces que ese número ha salido aleatoriamente.

Gráfica del histograma de números aleatorios, comentarios a la gráfica:

Hemos puesto que se generen 500000 números entre el 0 y el 20. Al final como resultado todos los números han aparecido aproximadamente 25000 veces. ($500000/20 = 25000$). Por lo tanto las probabilidades son bastante correctas, ya que en un baremo aproximado sale muy parecido.



5.2 Apartado 2

Tras ejecutar el exercise2.c el resultado obtenido debe dar numP arrays de tamaño size que vayan de 0 a size-1 y que hayan sido desordenados gracias a la llamada a la función del ejercicio 1. En este caso los array devueltos serán punteros.

5.3 Apartado 3

Ejecutando el exercise3.c el resultado obtenido debe ser numP arrays de tamaño size que van de 0 a size-1, que han sido desordenados gracias a la llamada a la función del apartado 2. En este caso, estos se guardarán en un doble puntero debido a que generate_perm devuelve punteros.

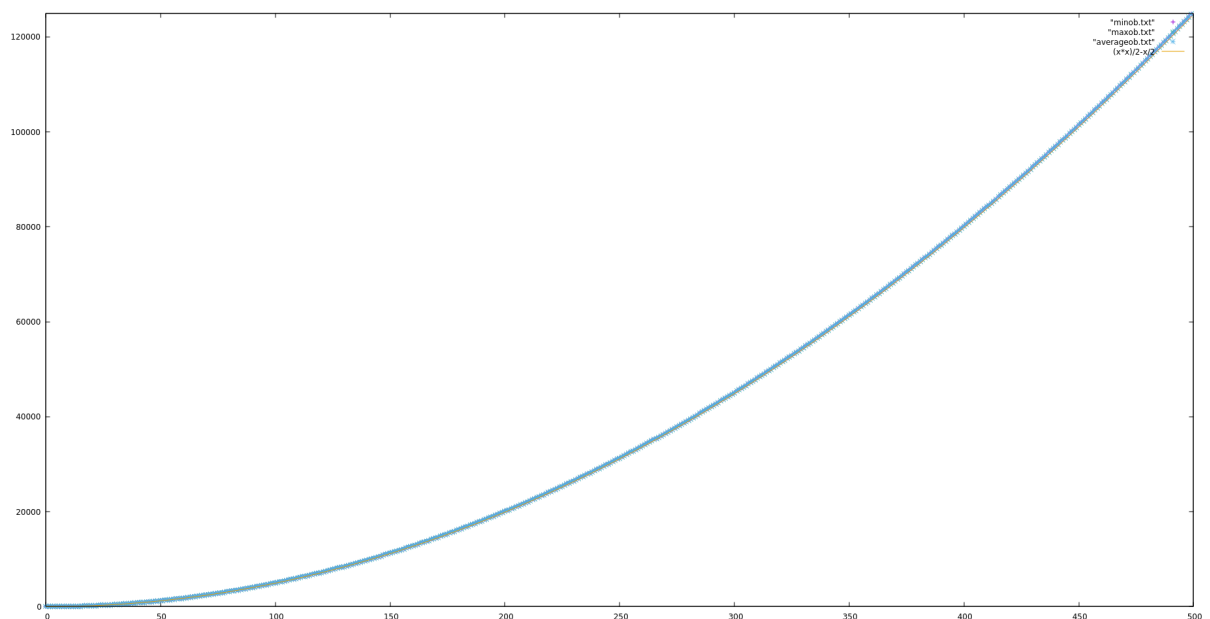
5.4 Apartado 4

En el apartado 4, se genera un array desordenado de un size determinado llamando a la función del apartado 3. Tras esto el exercise4.c llama a SelectSort para ordenar de menor a mayor este array. Por ejemplo, si el size es 10, aparecerá en la pantalla los números ordenados del 0 al 9 (de 0 a size-1).

5.5 Apartado 5

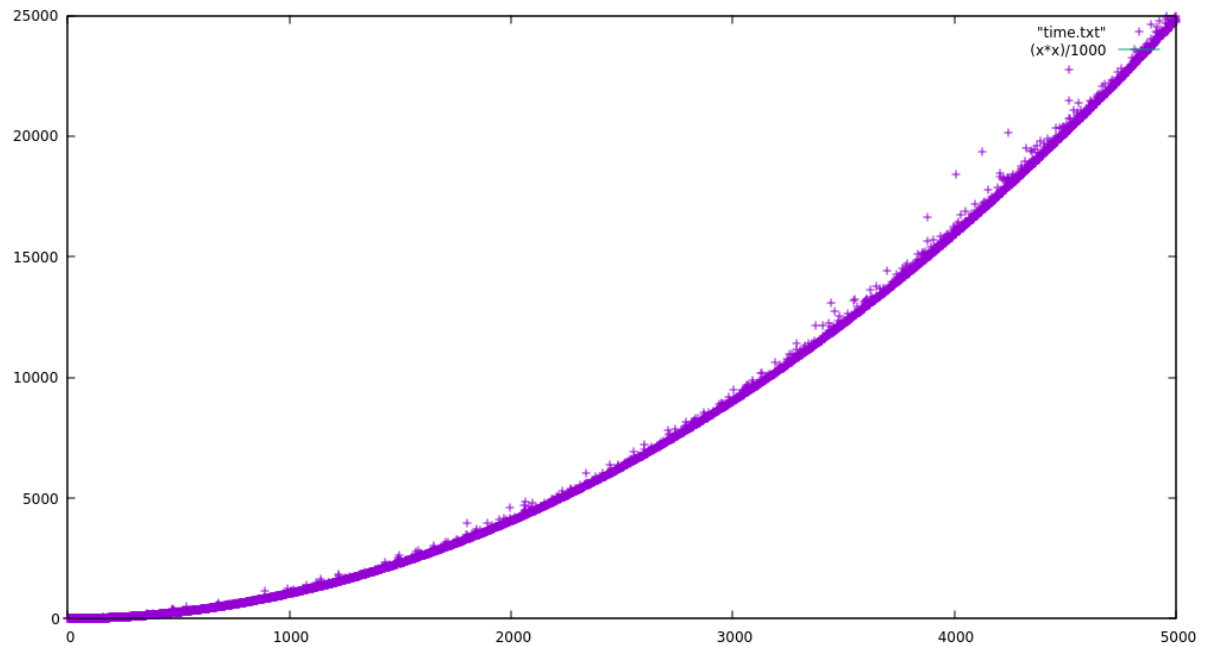
El apartado 5, devuelve un fichero en el cuál mide para cada array desordenado de tamaño size estos datos tras la llamada a SelectSort o SelectSortInv: el tiempo de ejecución del algoritmo, las veces que se ha ejecutado la operación básica para los casos mejor y peor y la media entre estos dos últimos datos. Este lo hace para sizes entre el num_min y el num_máx con un incremento incr. Si el número de veces que se repite el algoritmo es muy alto podemos apreciar en las gráficas que tanto el número de veces que se ejecuta OB para el caso peor y el mejor es el mismo y tiende a $N^2/2 - N/2$. Así como el tiempo, cuya gráfica también es similar a esta función.

Gráfica comparando los tiempos mejor, peor y medio en OBs para SelectSort, comentarios a la gráfica.



Cómo SelectSort siempre hace el mismo número de operaciones básicas todas las funciones son iguales en la gráfica. La complejidad es $N^2/2 - N/2$.

Gráfica con el tiempo medio de reloj para SelectSort, comentarios a la gráfica.

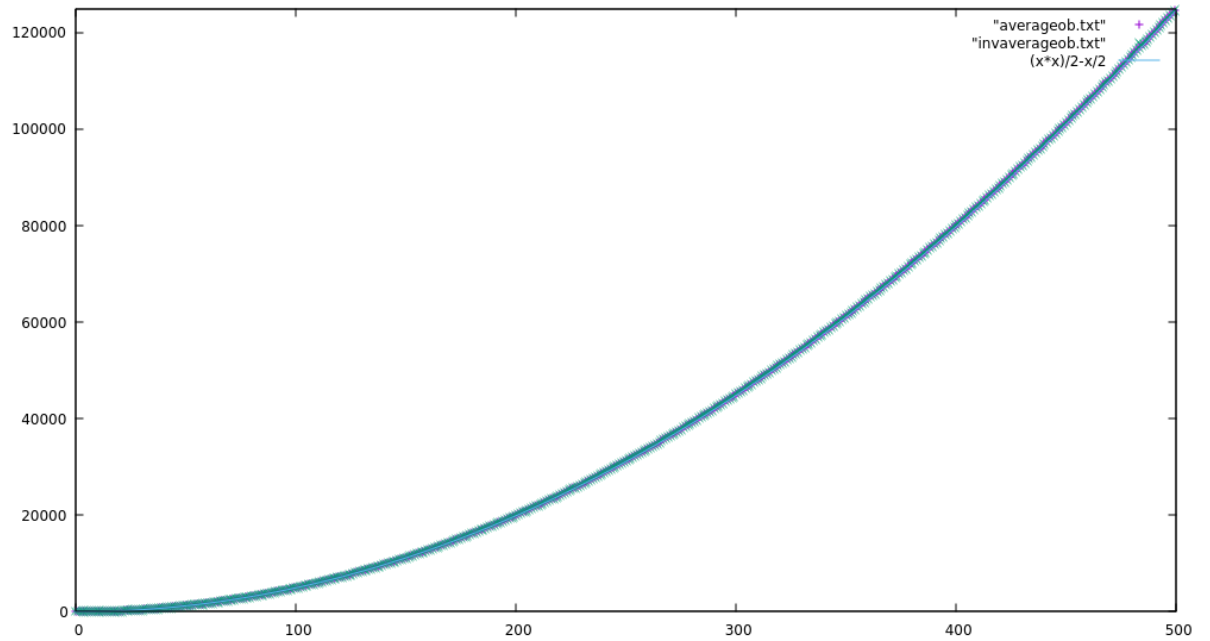


El tiempo medio también tiene la misma complejidad y al compararlo con x^2 vemos que nos da bien. x^2 lo dividimos entre 1000 ya que esta en milisegundos.

5.6 Apartado

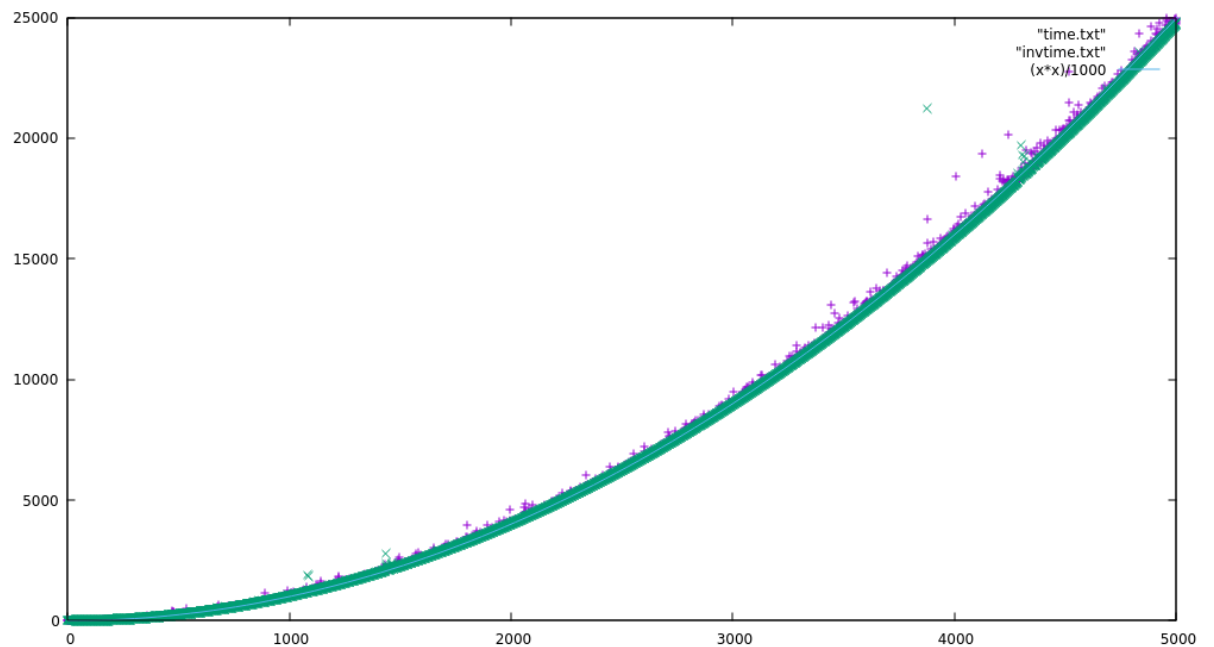
En el apartado 6, se genera un array desordenado de un size determinado llamando a la función del apartado 3. Tras esto el exercise4.c llama a SelectSortInv para ordenar de mayor a menor este array. Por ejemplo, si el size es 10, aparecerá en la pantalla los números ordenados del 9 al 0 (de size-1 a 0). En caso de querer ver los tiempos de ejecución y se hará mediante el exercise5.c. Podemos ver que como ambos bucles tanto externos como internos son iguales, SelectSort y SelectSortInv tienen el mismo tiempo de ejecución y sus OBs para el caso mejor y el peor se repite las mismas veces, pues siguen la misma función en la gráfica.

Gráfica comparando el tiempo medio de OBs para SelectSort y SelectSortInv, comentarios a la gráfica.



Las gráficas nos dan iguales ya que la complejidad de las dos funciones es la misma.

Gráfica comparando el tiempo medio de reloj para SelectSort y SelectSortInv, comentarios a la gráfica.



Este caso es el mismo que el anterior también da lo mismo.

6. Respuesta a las preguntas teóricas.

6.1 Pregunta 1

Para la implementación de la función `aleat_num` hemos usado la función `rand()`. Esta función, cada vez que la llamamos, nos devuelve un número entero aleatorio entre dos números ($n = \text{rand}() \% (\text{sup-inf}) + \text{inf}$). Con el `(sup-inf)` obtenemos el rango de los números que se van a generar y con el `+inf` obtenemos a partir de qué número se establece ese rango. Otra forma de generar números random es implementando nosotros nuestra propia función `rand`. Por ejemplo:

```
unsigned long next=1;

int rand(void) /* NOT RECOMMENDED (see text) */
{
    next = next*1103515245 + 12345;

    return (unsigned int) (next/65536) % 32768;
}

void srand(unsigned int seed)
{
    next=seed;
}
```

También podríamos cambiar el valor que le damos a la seed aunque sería menos efectivo.

6.2 Pregunta 2

El algoritmo `SelectSort` ordena bien ya que recorre la tabla buscando el número más bajo para ponerlo al principio. Y esto lo repite con todas las posiciones del array de 0 a N-1 , quedando esta totalmente ordenada al finalizar.

6.3 Pregunta 3

El bucle exterior no actúa sobre el último elemento del array ya que se ha recorrido toda la tabla N-1 veces encontrando el mínimo número que quedaba detrás de la posición que se buscaba. Esto hace que cuando el primer bucle haya recorrido todos los números menos el

último, en este se quede el mayor número del array, ya que en ningún momento ha podido ser el número más pequeño del array recorrido, y por lo tanto no es necesario compararlo. Además al ser el último, este no tiene elementos después de él por lo que no podrá compararse con otro valor.

6.4 Pregunta 4

La operación básica del SelectSort es la comparación de si un número es mayor que otro con el cual se ve cual es el menor elemento del array, ya que es el que está en el bucle más interno y es el que más veces se repite.

6.5 Pregunta 5

Los tiempos de ejecución del algoritmo para su caso mejor y su caso peor van a ser los mismos siempre, ya que la complejidad del algoritmo no se basa en su desorden, esta siempre será la misma. Esto quiere decir que para dos tablas ordenadas de forma distinta, ambas con N números, el algoritmo tardará lo mismo para ambas ($N^2/2 - N/2$). Por lo tanto:

$$W_{BS}(n) = O B_{BS}(n) \quad B_{BS}(n) = O W_{BS}(n)$$

$$W_{BS}(n) = \Theta B_{BS}(n) \quad B_{BS}(n) = \Theta W_{BS}(n)$$

$$W_{BS}(n) = \Omega B_{BS}(n) \quad B_{BS}(n) = \Omega W_{BS}(n)$$

6.5 Pregunta 6

Para ambos algoritmos podemos ver una función de tiempo y de repeticiones de la OB muy similar. Esto se debe a que el funcionamiento de ambas es prácticamente el mismo, ambas recorren toda la tabla de la misma forma para buscar un número, SelectSort busca el más pequeño y SelectSortInv el más grande. En cuanto a el código, solo se cambia la comparación de un $<$ a un $>$.

7. Conclusiones finales.

Como conclusión podemos decir que ha sido una práctica bastante útil para conocer a fondo los algoritmos que hemos tenido que emplear y para aprender a generar gráficas y medir tiempos. Las funciones se han implementado con sus debidos controles de errores y hemos intentado que sean lo más eficientes posibles. Los resultados en principio son correctos y las gráficas generadas nos dan bien, ya que las hemos comparado con la gráfica de la función que deberían dar y son iguales.